

Digital Slot Machine

COMPE475 - Final Report

Wednesday, April 23th, 2025

Author: Rhilo Sotto

Red ID: 130551574

High Level Description:

For my project, I implemented a digital slot machine by using LFSRs to generate random numbers, which I convert into symbols through a NumToSymbol module, which is passed to a score calculator to determine whether the user wins. This score is multiplied by a BetController module which determines the user's score based on how large a bet was placed. The FPGA will have four buttons, one will act as the input to spin the reels, another to reset the machine, and two buttons to increase or decrease betting amounts. Score is outputted as a 16-bit array.

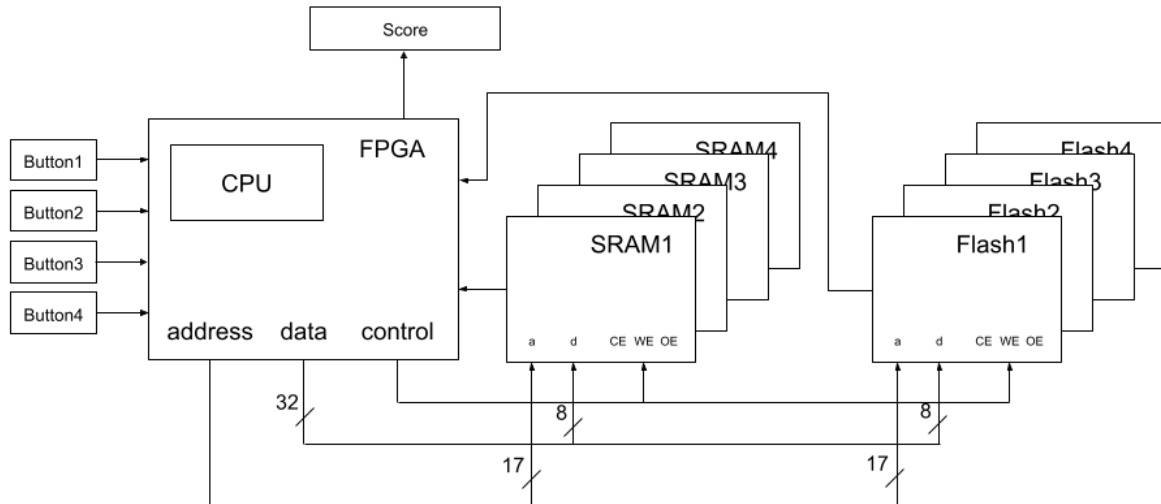
For this project, I am using the Artix 7 FPGA, 4 SRAM modules, 4 Nor Flash modules, 4 push buttons, and the MicroBlaze for the processor. Below are the links to the data sheets of each IC.

Data Sheets:

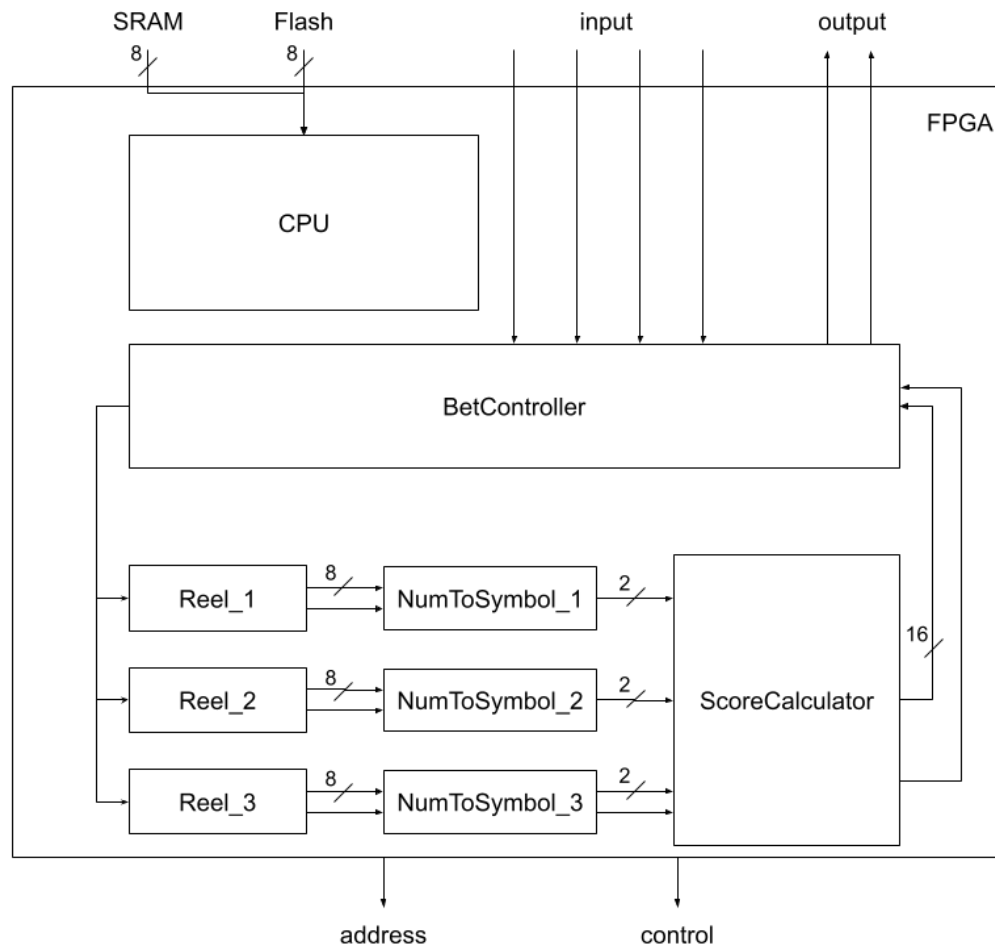
- FPGA (Artix-7): [XC7A35T-1CPG236C](#)
- SRAM: [IS63WV1024BLL-12TLI](#)
- Flash: [S29AL008J70TFI020](#)
- CPU: [MicroBlaze](#)

Block Diagrams:

High Level Design:



Top Level FPGA:



Memory Map:

<https://docs.google.com/spreadsheets/d/1pGU-33OcWKWroxhi46INQ6LN4t0JqcqhIAYuz1fPEmo/edit?gid=876070175#gid=876070175>

Component	Start Address	End Address	Size
SRAM	0x00000000	0x0003FFFF	128KB
Flash	0x00040000	0x0013FFFF	1MB
FPGA	0x00140000	0x1013FFFF	256MB
Input (4 buttons)	0x10140000	0x1FFFFFFF	4 bits
Output (Score)	0x20000000	0x2FFFFFFF	16 bits
Unused	0x30000000	0xFFFFFFFF	x

Noise Margin:

<https://docs.google.com/document/d/1uDE0xPGrFuNJ29455wqFcYvyu3SG075bWXjzS6eyu-s/e/dit?tab=t.63xehog154hj#heading=h.mfg062w8g7n>

Output					Input				Noise Margin	
Signal	Pin(s)	Source	Vol max	Voh min	Load(s)	Signal	Vil max	Vih min	logic zero	logic one
RD		FPGA	0.4	2.9	SRAM	OE	0.8	2	0.4	0.9
			0.4	2.9	NOR Flash	OE	0.8	2.1	0.4	0.8
CE		FPGA	0.4	2.9	SRAM	CE	0.8	2	0.4	0.9
			0.4	2.9	NOR Flash	CE	0.8	2.1	0.4	0.8
WR		FPGA	0.4	2.9	SRAM	WR	0.8	2	0.4	0.9
			0.4	2.9	NOR Flash	WR	0.8	2.1	0.4	0.8
Address	A 0-18	FPGA	0.4	2.9	SRAM	A0-15	0.8	2	0.4	0.9
			0.4	2.9	NOR Flash	A18-A0	0.8	2.1	0.4	0.8
Data	D 0-7	SRAM	0.4	2.4	FPGA	LVCMO S33	0.8	2	0.4	0.4
			0.4	2.4	NOR Flash	D7-D0	0.8	2.1	0.4	0.3
		NOR Flash	0.45	2.55	FPGA	LVCMO S33	0.8	2	0.35	0.55
			0.45	2.55	SRAM	D0-D15	0.8	2	0.35	0.55
		FPGA	0.4	2.9	SRAM	D0-D15	0.8	2	0.4	0.9
			0.4	2.9	NOR Flash	D7-D0	0.8	2.1	0.4	0.8

Loading Analysis:

<https://docs.google.com/spreadsheets/d/1pGU-33OcWKWroxhi46INQ6LN4t0JqcqhIAYuz1fPEm0/edit?gid=1961246309#gid=1961246309>

SOURCE			uA	uA	pF	LOAD		Unit Load				Total		
Signal	PIN	Source	IOL	IOH	CL	Load	Signal	Qty	IIL (uA)	IIH (uA)	Cin (pF)	IIL (uA)	IIH (uA)	Cin (pF)
D0-32		FPGA	8000	-8000	0	SRAM	I/O 0-7	1	-2	2	6	-2	2	6
						FLASH	DQ 0-7	1	-1	10	6	-1	10	6
						wire cap		3			2			6
											Total	-3	12	18
											Margin	7997	7988	-18
		SRAM	8000	-4000	30	FPGA	DQ 0-7	1	-15	15	8	-15	15	8
						FLASH	DQ 0-7	1	-1	10	12	-1	10	12
						wire cap		3			2			6
											Total	-16	25	26
											Margin	7984	3975	4
		NOR FLASH	4000	-100	30	SRAM	I/O 0-7	1	-2	2	6	-2	2	6
						FPGA	DQ 0-7	1	-15	15	8	-15	15	8
						wire cap		3			2			6
											Total	-17	17	20
											Margin	3983	83	10
A0-A18		FPGA	8000	-8000	0	SRAM	A0-A18	4	-2	2	6	-8	8	24

						FLASH	A0-A18	4	-1	10	6	-4	40	24
						wire cap		3			2			6
											Total	-12	48	54
											Margin	7988	7952	-54
CE		FPGA	8000	-8000	0	SRAM	CE	1	-2	2	6	-2	2	6
						FLASH	CE	1	-1	10	6	-1	10	6
						wire cap		3			2			6
											Total	-3	12	18
											Margin	7997	7988	-18
OE		FPGA	8000	-8000	0	SRAM	OE	1	-2	2	6	-2	2	6
						FLASH	OE	1	-1	10	6	-1	10	6
						wire cap		3			2			6
											Total	-3	12	18
											Margin	7997	7988	-18
WE		FPGA	8000	-8000	0	SRAM	WE	1	-2	2	6	-2	2	6
						FLASH	WE	1	-1	10	6	-1	10	6
						wire cap		3			2			6
											Total	-3	12	18
											Margin	7997	7988	-18

Timing Analysis:

<https://docs.google.com/spreadsheets/d/1pGU-33OcWKWroxhi46INQ6LN4t0JqcqhlAYuz1fPEm/o/edit?gid=1291085787#gid=1291085787>

Signals	Direction	Datasheet Spec (ns)	Delta V (V)	Delta C (pF)	Delta I	Delta T	Derated Value
CE	CPU to SRAM	12	1.2	2	3	0.8	12.8
OE		6	1.2	2	3	0.8	6.8
WE		12	1.2	2	3	0.8	12.8
A0-A18		9	1.2	2	3	0.8	9.8
D0-D31		9	1.2	18	3	7.2	16.2
CE	CPU to Flash	55	1.2	2	3	0.8	55.8
OE		30	1.2	2	3	0.8	30.8
WE		55	1.2	2	3	0.8	55.8
A0-A18		45	1.2	2	3	0.8	45.8
D0-D31		35	1.2	18	3	7.2	42.2

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.948 ns	Worst Hold Slack (WHS): 0.122 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 116	Total Number of Endpoints: 116	Total Number of Endpoints: 70

All user specified timing constraints are met.

Bitstream Timing Report: WNS + WHS + WPWS + Clock Period = 5.948 + 0.122 + 4.500 + 10.00 = 20.57ns

WNS	WHS	WPWS	Clock Period	Total (ns)
5.948	0.122	4.5	10	20.57

Total Cycle Time		
Source	Read (ns)	Write (ns)
CPU to SRAM	40.17	46.17
CPU to Flash	107.17	132.17

Function Description

Top Module:

The Top module utilizes the Reel, NumToSymbol, ScoreCalculator, and BetController. The three reels are triggered when the button (BTN) is pressed and the signal valid is sent from BetController, which is only sent when the user set bet amount is less than or equal to the amount of score they currently possess. The number generated by the reels is sent to be translated into a corresponding symbol which has custom probabilities set in NumToSymbol. The three symbols are checked whether they are equal within the ScoreCalculator, which sends out the base score the user would receive back to BetController to be multiplied and give the player their final score for that spin.

Reel:

The Reel utilizes an 8 bit LFSR with tap bits on 8, 6, 5, and 4 for a maximal length sequence on a seed value defined as a parameter of the module. The reel will spin 4 times before outputting a number alongside a signal indicating the process is finished.

NumToSymbol:

The NumToSymbol takes in an 8 bit number from the Reel module, and maps the number to a range of values. The range the number lands in determines the symbol that is outputted.

ScoreCalculator:

The ScoreCalculator accepts three symbols (2 bit signals) and checks if all three symbols are equal, and if so, generates a score corresponding to the matching symbols. Rarer symbols give a larger amount of score compared to more common symbols. If all three symbols do not match, the score generated is zero.

BetController:

The BetController contains a 16 bit register for the user's overall score, which it will deduct the bet amount from when the user spins the wheel. Depending on the base score the module receives, the amount will be multiplied by whatever bet amount the user set and then added back to the user's overall score. The BetController also controls the validity of the user's bet, so if the user attempts to spin while having a bet amount higher than their current score total, then the controller will not output a valid signal to start the reels.

SRAM:

The SRAM is able to read and write data. If the CE and OE signals are off then data can be read from the SRAM. If the CE and WE signals are off then data can be written into the memory.

Flash:

The Flash is similar to the SRAM, and can do the same with the CE, OE, and WE signals.

Files for running code:

No external files are necessary for running my code.

Tools Utilized:

The tool I used to run my project was Vivado.

Code, Test Bench, and Simulation:

Top Module:

```
module top(
    input CLK,
    input RST,
    input BTN,
    input BTN_U, BTN_D,
    output wire [15:0] Score,

    output wire done_r1, done_r2, done_r3,
    output wire done_ns1, done_ns2, done_ns3,
    output wire done_sc,
    output wire done_bc,
    output wire valid,

    output wire [7:0] num_1, num_2, num_3,
    output wire [1:0] symbol_1, symbol_2, symbol_3,
    output wire [3:0] score_sc,
    output wire [3:0] bet,
    output wire [7:0] score_bc
);

// wire done_r1, done_r2, done_r3;
// wire done_ns1, done_ns2, done_ns3;
// wire done_sc;
// wire done_bc;
// wire valid;

// wire [7:0] num_1, num_2, num_3;
// wire [1:0] symbol_1, symbol_2, symbol_3;
// wire [3:0] score_sc;
// wire [7:0] score_bc;

    Reel #(.SEED(8'b10101000)) R1(.CLK(CLK), .RST(RST), .spin(valid), .num(num_1),
.done(done_r1));
    Reel #(.SEED(8'b10101010)) R2(.CLK(CLK), .RST(RST), .spin(valid), .num(num_2),
.done(done_r2));
    Reel #(.SEED(8'b11101000)) R3(.CLK(CLK), .RST(RST), .spin(valid), .num(num_3),
.done(done_r3));
```

```
    NumToSymbol NS1 (.CLK(CLK), .convert(done_r1), .num(num_1), .symbol(symbol_1),  
    .done(done_ns1));
```

```
    NumToSymbol NS2 (.CLK(CLK), .convert(done_r2), .num(num_2), .symbol(symbol_2),  
    .done(done_ns2));
```

```
    NumToSymbol NS3 (.CLK(CLK), .convert(done_r3), .num(num_3), .symbol(symbol_3),  
    .done(done_ns3));
```

```
    ScoreCalculator SC(  
        .CLK(CLK),  
        .calculate(done_ns3),  
        .Symbol_1(symbol_1), .Symbol_2(symbol_2), .Symbol_3(symbol_3),  
        .Score(score_sc),  
        .done(done_sc)  
    );
```

```
    BetController BC(  
        .CLK(CLK),  
        .RST(RST),  
        .BTN(BTN),  
        .bet_up(BTN_U), .bet_down(BTN_D),  
        .calculate(done_sc),  
  
        .base(score_sc),  
  
        .valid(valid),  
  
        .Score(Score),  
  
        .bet(bet),  
        .score_out(score_bc),  
  
        .done(done_bc)  
    );
```

```
endmodule
```

tb_Top:

```
module tb_top;
```

```
reg CLK = 0;
  always begin #0.5; CLK <= ~CLK; end
reg RST = 0;
reg BTN = 0;
wire [15:0] score;
reg BTN_U = 0, BTN_D = 0;
```

```
wire done_r1, done_r2, done_r3;
wire done_ns1, done_ns2, done_ns3;
wire done_sc;
wire done_bc;
wire valid;
```

```
wire [7:0] num_1, num_2, num_3;
wire [1:0] symbol_1, symbol_2, symbol_3;
wire [3:0] score_sc;
wire [3:0] bet;
wire [7:0] score_bc;
```

```
top TOP(
  .CLK(CLK),
  .RST(RST),
  .BTN(BTN),
  .BTN_U(BTN_U), .BTN_D(BTN_D),
  .Score(score),

  .done_r1(done_r1), .done_r2(done_r2), .done_r3(done_r3),
  .done_ns1(done_ns1), .done_ns2(done_ns2), .done_ns3(done_ns3),
  .done_sc(done_sc),
  .done_bc(done_bc),
  .valid(valid),

  .num_1(num_1), .num_2(num_2), .num_3(num_3),
  .symbol_1(symbol_1), .symbol_2(symbol_2), .symbol_3(symbol_3),
  .score_sc(score_sc),
  .bet(bet),
  .score_bc(score_bc)
);
```

```
initial begin
```

```
    @(posedge CLK)
    BTN = 1;
    @(posedge CLK)
    BTN = 0;
    repeat(10) @(posedge CLK);
```

```
// reset
    @(posedge CLK)
    RST = 1;
    @(posedge CLK)
    RST = 0;
```

```
// spin
repeat(5) begin
    @(posedge CLK)
    BTN = 1;
    @(posedge CLK)
    BTN = 0;
    repeat(10) @(posedge CLK);
end
```

```
// increase bet amount +2
    @(posedge CLK)
    BTN_U = 1;
    repeat(2) @(posedge CLK);
    BTN_U = 0;
// spin
repeat(5) begin
    @(posedge CLK)
    BTN = 1;
    @(posedge CLK)
    BTN = 0;
    repeat(10) @(posedge CLK);
end
```

```
// decrease bet amount -2
    @(posedge CLK)
    BTN_D = 1;
```

```
repeat(2) @(posedge CLK);
BTN_D = 0;
// spin
repeat(5) begin
    @(posedge CLK)
    BTN = 1;
    @(posedge CLK)
    BTN = 0;
    repeat(10) @(posedge CLK);
end

$finish;
end

endmodule
```

Reel:

```
module Reel #(parameter SEED = 8'b00000001) (  
    input CLK,  
    input RST,  
    input spin,  
    output reg [7:0] num = SEED,  
    output reg done = 0  
);  
  
// States  
localparam Idle = 2'b00, Spinning = 2'b01, Done = 2'b10;  
reg [1:0] state = Idle;  
// how many times to cycle before stopping  
reg [3:0] spin_count;  
  
always @(posedge CLK or posedge RST) begin  
    if(RST) begin // reset signal  
        state <= Idle;  
        done <= 0;  
        spin_count <= 4'b0000;  
        num <= SEED;  
    end  
    else begin  
        case(state)  
            Idle: begin  
                done <= 0;  
                if(spin) begin  
                    state <= Spinning;  
                    spin_count <= 4'b0000;  
                end  
            end  
            Spinning: begin  
                if(spin_count < 4'b0100) begin // 8 Bit LFSR with tap bits at 8, 6, 5, 4 for maximal  
length  
                    num <= {num[6], num[5], num[4], num[3], num[2], num[1], num[0],  
num[7]^num[5]^num[4]^num[3]};  
                    spin_count <= spin_count + 1;  
                end  
            end  
            else begin  
                state <= Idle;  
                done <= 1;  
            end  
        end  
    end  
end
```

```
        end
    endcase
end
end
endmodule
```

tb_Reel:

```
`timescale 1ns / 1ps
```

```
module tb_reel;
```

```
    reg CLK = 0;  
    always begin #2; CLK <= ~CLK; end
```

```
    reg RST = 0;  
    reg spin = 0;  
    wire [7:0] num;  
    wire done;
```

```
    Reel #(.SEED(8'b10101000)) R1(.CLK(CLK), .RST(RST), .spin(spin), .num(num),  
    .done(done));
```

```
    initial begin  
        @(posedge CLK)  
        spin = 1;  
        @(posedge CLK)  
        spin = 0;  
        repeat(5) @(posedge CLK);
```

```
        @(posedge CLK)  
        RST = 1;  
        @(posedge CLK)  
        RST = 0;
```

```
        repeat(5) begin  
            @(posedge CLK)  
            spin = 1;  
            @(posedge CLK)  
            spin = 0;  
            repeat(5) @(posedge CLK);  
        end
```

```
    $finish;  
end
```

```
endmodule
```

NumToSymbol:

```
module NumToSymbol(
    input CLK,
    input convert,
    input [7:0] num,
    output reg [1:0] symbol,
    output reg done
);

// 4 symbols with varying probabilities
localparam A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

// 256 possible nums
always @(posedge CLK) begin
    if(convert) begin
        if(num < 128) begin // 50%
            symbol <= A;
        end
        else if(num < 128 + 64) begin // 25%
            symbol <= B;
        end
        else if(num < 128 + 64 + 32 + 16) begin // 12.5% + 6.25%
            symbol <= C;
        end
        else begin // 6.25%
            symbol <= D;
        end
        done <= 1;
    end
    else begin
        done <= 0;
    end
end

endmodule
```

ScoreCalculator:

```
module ScoreCalculator(  
    input CLK,  
    input calculate,  
    input [1:0] Symbol_1, Symbol_2, Symbol_3,  
    output reg [3:0] Score,  
    output reg done  
);  
  
    // 4 symbols with varying probabilities  
    localparam A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;  
  
    always @(posedge CLK) begin  
        if(calculate) begin  
            if ((Symbol_1 == Symbol_2) && (Symbol_2 == Symbol_3)) begin  
                case(Symbol_3) // all three are the same, payout based on rarity  
                    A: Score <= 1;  
                    B: Score <= 3;  
                    C: Score <= 5;  
                    D: Score <= 15;  
                endcase  
            end  
            else begin  
                Score <= 0; // at least one reel is different, lose  
            end  
            done <= 1;  
        end  
        else begin  
            done <= 0;  
        end  
    end  
end  
  
endmodule
```



```

        bet <= ((bet < 1) || (bet == 1)) ? bet : bet - 1;
    end
end
if(BTN) begin // check if user has enough score to spin
    if(Score < bet) begin
        valid <= 0;
    end
    else begin
        Score <= Score - bet;
        valid <= 1;
        state <= Spinning;
    end
end
end
Spinning: begin
    valid <= 0; // wait for base score to be calculated by scorecalculator
    if(calculate) begin
        score_out <= bet * base;
        Score <= Score + bet * base;
        done <= 1;
        state <= Idle;
    end
end
end

endcase
end
end

endmodule

```

Constraint File

Clock signal

```
set_property PACKAGE_PIN W5 [get_ports CLK]
set_property IOSTANDARD LVCMOS33 [get_ports CLK]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK]
```

Score pins

```
set_property PACKAGE_PIN U16 [get_ports {Score[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[0]}]
set_property PACKAGE_PIN E19 [get_ports {Score[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[1]}]
set_property PACKAGE_PIN U19 [get_ports {Score[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[2]}]
set_property PACKAGE_PIN V19 [get_ports {Score[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[3]}]
set_property PACKAGE_PIN W18 [get_ports {Score[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[4]}]
set_property PACKAGE_PIN U15 [get_ports {Score[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[5]}]
set_property PACKAGE_PIN U14 [get_ports {Score[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[6]}]
set_property PACKAGE_PIN V14 [get_ports {Score[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[7]}]
set_property PACKAGE_PIN V13 [get_ports {Score[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[8]}]
set_property PACKAGE_PIN V3 [get_ports {Score[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[9]}]
set_property PACKAGE_PIN W3 [get_ports {Score[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[10]}]
set_property PACKAGE_PIN U3 [get_ports {Score[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[11]}]
set_property PACKAGE_PIN P3 [get_ports {Score[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[12]}]
set_property PACKAGE_PIN N3 [get_ports {Score[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[13]}]
set_property PACKAGE_PIN P1 [get_ports {Score[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[14]}]
set_property PACKAGE_PIN L1 [get_ports {Score[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Score[15]}]
```

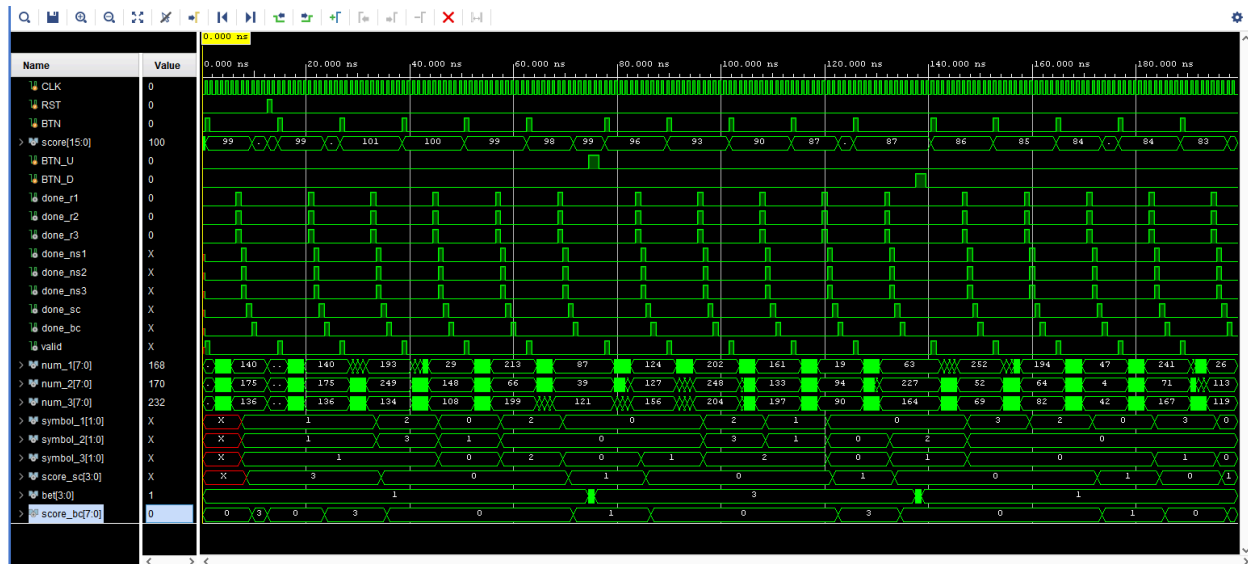
#Buttons

```
set_property PACKAGE_PIN W19 [get_ports {BTN}]
    set_property IOSTANDARD LVCMOS33 [get_ports {BTN}]
set_property PACKAGE_PIN U18 [get_ports {RST}]
    set_property IOSTANDARD LVCMOS33 [get_ports {RST}]

set_property PACKAGE_PIN T18 [get_ports {BTN_U}]
    set_property IOSTANDARD LVCMOS33 [get_ports {BTN_U}]
set_property PACKAGE_PIN U17 [get_ports {BTN_D}]
    set_property IOSTANDARD LVCMOS33 [get_ports {BTN_D}]
```

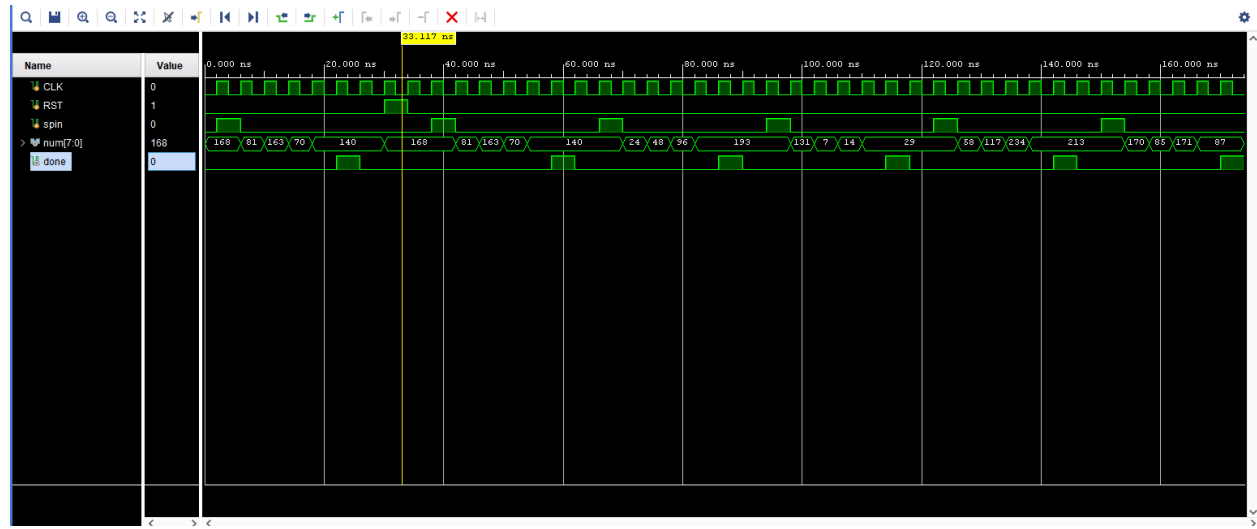
Waveforms:

Top Module



The machine is working as expected, with bet amount being detracted from score on every spin (BTN pulse). To walk through the process, whenever BTN goes up, the betcontroller module checks to verify if the amount of score is greater than or equal to the bet amount, if so, then the spin is considered valid and activates the three reels. The three reels roll three times (not as obvious in this waveform) , and deliver a signal when their processing is completed (done_r#), alongside their respective numbers. This is sent to their respective numbertosymbol converter, which will take the raw number and determine which symbol should be outputted as well as their complete signal (done_ns#). After all three are done computing they will send the symbol to the score calculator, which will determine how much the user wins (done_sc). This is sent back to the betcontroller to multiply the base score amount by the bet amount and add this back to the user's score total (done_bc). I first showcase an initial spin, the reset feature (RST), and the increase (BTN_U) and decrease (BTN_D) bet buttons.

Reel Module



This module works as expected, the reel spins through three numbers before landing on the fourth, which is sent out alongside the done signal. I designed it in this way to interface with an BCD for an LED display, which would simulate the slot reels physically spinning and cycling through random numbers. The reel in this testbench uses the seed 8'b10101000, the top level module uses two other reels with different seeds to achieve a more random result.

What I would do differently:

There are a couple more features I would have liked to be able to implement: a button debounce module for each of the input buttons (BTN, BTN_U, BTN_D, RST), a BCD for an LED display output, which I ended up excluding for the sake of a more readable top module waveform. I would also have made individual testbenches for the NumToSymbol, ScoreCalculator, and BetController modules, but I also found it easier to exercise the top module as the design under test and view the signals between the submodules as outputs. The modules working together as one larger process made more sense to test and I found each module tested sufficiently.

Conclusion:

The process of planning, designing, and implementing this project has helped me understand how to go through datasheets for components. It taught me to be careful when picking components, especially considering the noise margins, loading analysis, and timing involved to ensure the validity of the design. This project made me appreciate the compartmentalization of modules in Verilog with how they all interact with each other, as well as the importance of naming conventions within multi-layer modules.

Hours Spent:

<https://docs.google.com/spreadsheets/d/1pGU-33OcWKWroxhi46INQ6LN4t0JqcqhIAYuz1fPEm/edit?gid=149641410#gid=149641410>

Module + tb	Short Description	Hours
Top	Connects all the modules	8
Reel	Generates a random number with an LFSR	4
NumToSymbol	Takes a random number and maps it to a symbol	5
ScoreCalculator	Takes three symbols and calculates the score	5
BetController	Subtracts bet from score and multiplies winnings by bet amount	6
SRAM	Volatile memory	3
Flash	Nonvolatile memory	3